Exercise Project X

Data Generator:

I tested with three different ranges of data and my observations were that with the baseline data range the loss starts at 2.9 and drops gradually to 2.24 which shows smooth and slow convergence which indicates stable learning. In the second case, I increased the range up to 20 and loss jumped aggressively from 91 to 11,468 and instead of decreasing started increasing which shows that the training completely failed. This happened because the learning rate 0.001 becomes too huge for this large input and causes gradient explosion. In the last case, I decreased the data range even further down then the baseline and that showed a loss decrease initially before starting to increase again which shows that the data has very low variation and too many similar samples which cause the Neural network to learn quickly what it can and then over adjusting weights.

Weights and Bias:

I experimented with random start weights and bias which produced the best results and converged smoothly. It happens because it breaks symmetry between neurons and leads to faster and better convergence. I also tested with all weights set to the same value which causes loss to start extremely high and plateaus early. In the last case I tested with weights set to zero which caused a high loss and stagnated training and no improvement.

Learning Rate:

I tested with various learning rates but it shows that the neural network is very simple and the data is too small and the learning rate very quickly becomes very big for it making the training break at even LR = 0.05 after a few early epochs.

Loss Plot:

The loss plot shows that the loss decreases in early epochs and improves slowly, eventually flattening. This shows that the Neural network learns the structure quickly and then fine-tunes the parameters. It converges at around 400 epochs where the curve goes close to zero and further epochs only have slight improvements.

Small Changes:

I observed that even a small change causes a drastic change in performance especially in a simple neural network because in a much more complex Neural network if a few neurons fail others might be able to compensate but in a simpler Neural network each neuron is emphasized more and change causes drastic effects to performance.

Neural networks Working:

I observed that a Neural network works by taking an input and passing it through the layers of the network and each step a weight value is assigned to the input and passed along with the result through activation function and in the end the network produces a prediction. The network then compares the prediction to the correct answer using a loss function. Using Backpropagation the network checks which weights led to the error and fine tunes it to improve the network.

Learning rate, Gradient descent and the Loss-function:

The loss function measures the difference between actual ground truth and network predictions. This value should be as small as possible and the network finetune itself to drop the value after each epoch ideally.

Gradient Descent is the algorithm that helps minimize loss function by calculating the slope of the loss function with respect to each weight

Learning Rate is the rate at which the network finetunes itself to minimize the loss; the learning rate defines how big of a step the network takes towards the negative descent in one epoch.

Loss function defines how the network is performing, Gradient descent is the method that reduces loss and Learning rate is the rate at which the loss is reduced.